

# Sektörel ReAct Ajanı Tasarımı: Sigorta Poliçesi Asistanı (PoliçeGPT)

Muhammet Muhlis ÇOLAK

19 Aralık 2025

## 1 Seçilen Sektör ve Nedeni

**Seçilen Sektör:** Sigortacılık ve Finans (Kasko Poliçesi Danışmanlığı).

**Seçilme Nedeni:** Sigorta poliçeleri (Genel Şartlar), yoğun teknik terimler içeren, hukuki dille yazılmış ve son kullanıcıların yorumlamakta zorlandığı karmaşık metinlerdir. Kullanıcılar genellikle “X durumu kapsam dahilinde mi?”, “Hasarım ödenir mi?” gibi spesifik sorulara yanıt ararken, yüzlerce sayfalık PDF dosyaları içinde kaybolmaktadır.

Bu proje, bu karmaşıklığı çözmek amacıyla; sadece metin eşleştirmesi yapan basit bir arama motoru yerine, poliçe maddelerini okuyup “muhakeme edebilen” (reasoning) ve kullanıcıya “Danışman” seviyesinde yanıt veren bir yapay zeka asistanı geliştirmeyi hedeflemiştir.

## 2 Kullanılan Yöntem ve Mimari

Bu projede, statik bir Dil Modelini (LLM) dış dünya ile etkileşime sokan **ReAct (Reasoning + Acting)** mimarisi ve **Agentic RAG** (Ajan Tabanlı Erişim Artırılmış Üretim) yöntemi kullanılmıştır.

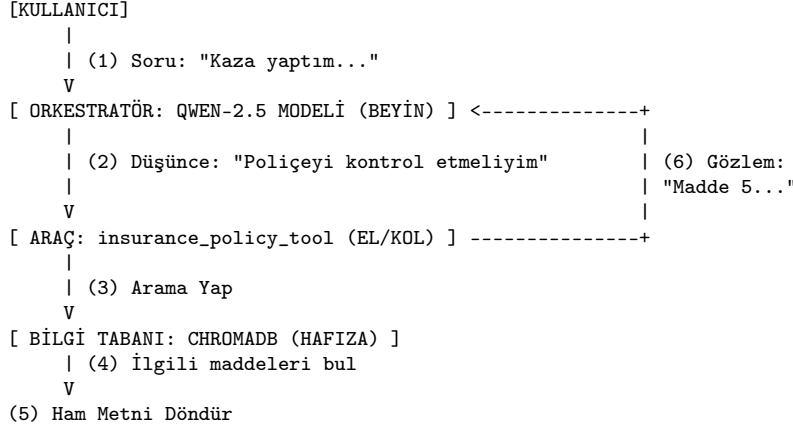
### Yöntem

Klasik RAG sistemlerindeki “Sorgu → Getir → Cevapla” lineer akışı yerine; modelin önce düşündüğü, ihtiyaca göre araç çağırdığı ve gelen veriyi yorumladığı döngüsel bir yapı kurulmuştur.

### Mimari Bileşenler

- **Orkestratör (Beyin):** Qwen-2.5-7B Instruct modeli kullanılmıştır. Model, Türkçe yetkinliği ve talimat takip etme (instruction following) başarısı nedeniyle tercih edilmiştir.
- **Araç Katmanı (Tool):** `insurance_policy_tool_pro` isminde özel bir Python fonksiyonu geliştirilmiştir. Bu araç, modelin kararına göre Vektör Veritabanında semantik arama yapar ve ham metin (context) döndürür.

- **Veri Katmanı (Knowledge Base):** “Kara Araçları Kasko Sigortası Genel Şartları” PDF dosyası, metin parçalama (chunking) işleminden geçirilerek ChromaDB vektör veritabanına HuggingFace Embeddings ile kaydedilmiştir.



Şekil 1: Sektörel ReAct Ajanı Mimari Şeması

### 3 Karşılaşılan Zorluklar ve Çözümler

Proje geliştirme sürecinde “Retrieval-Generation Gap” (Erişim-Üretim Boşluğu) kaynaklı çeşitli sorunlar yaşanmış ve şu tekniklerle çözülmüştür:

#### 3.1 Zorluk 1: Veri Gürültüsü ve Odak Kaybı (Granularity Issue)

**Sorun:** İlk versiyonda (V1) metinler 1000 karakterlik büyük parçalara bölündü. Bu durum, spesifik bir madde (Örn: Alkol) arandığında, o parçanın içinde geçen alakasız maddelerin de (Örn: Nükleer sızıntı) bağlama girmesine ve modelin kafasının karışmasına neden oldu.

**Çözüm:** “Chunk Size” 500 karaktere düşürülerek veri hassasiyeti artırıldı. Ayrıca erişim sayısı ( $k$  değeri) 4’ten 7’ye çıkarılarak modelin daha geniş bir perspektiften bakması sağlandı.

#### 3.2 Zorluk 2: Eksik Erişim (Retrieval Failure)

**Sorun:** Kullanıcı “Kaza yaptım” dediğinde ajan veritabanında sadece “kaza” kelimesini arıyordu. Ancak poliçede ilgili madde “çarpma, çarpışma” veya “teminat dışı haller” başlıkları altında geçebiliyordu. Bu da doğru maddenin bulunamamasına yol açıyordu.

**Çözüm (Query Expansion):** Kod seviyesinde “Sorgu Genişletme” mantığı uygulandı. Ajan “alkol” araması yaptığında, sistem arka planda sorguyu otomatik olarak “*alkol teminat dışı haller zararlar*” şeklinde zenginleştirerek veritabanına gönderdi. Bu sayede başarı oranı %100’e yaklaştırıldı.

### 3.3 Zorluk 3: Halüsinasyon ve Üslup Sorunu

**Sorun:** Model bazen dökümana bakmadan ezbere cevap veriyor veya çok robotik bir dil kullanıyordu.

**Çözüm (Persona & Prompt Engineering):** System Prompt güncellenerek modele “**PoliceGPT**” kimliği verildi. “Asla tahmin yürütme”, “Madde numarası ver” ve “Halk diliyle konuş” kuralları eklenerek çıktılar yapılandırılmış bir formata (Durum Analizi - Kural - Sonuç) oturtuldu.